

DEEP REINFORCEMENT LEARNING

365.382 (SS26)

Core Challenges in DRL



Sohvi Luukkonen

LIT AI LAB

Institute for Machine Learning

Recap and Motivation

Recap: How Did We Get Here? (1/2)

1. Tabular RL (Q-learning, SARSA)

- ☐ Strong theoretical guarantees (convergence, optimality)
- ☐ Does not scale to large or continuous state spaces
- ☐ (Quasi-)deterministic greedy policies: $a = \arg \max_a Q(s, a)$

2. Value Function Approximation (DQN + extensions)

- ☐ Generalizes across large or continuous state spaces via neural networks
- ☐ Struggles with large or continuous action spaces (argmax over actions is intractable)
- ☐ Policy remains (quasi-)deterministic: $a = \arg \max_a Q(s, a)$

Recap: How Did We Get Here? (2/2)

3. Policy Gradient (REINFORCE)

- ☐ Directly parameterizes the policy π_θ – true objective
- ☐ Handles continuous actions
- ☐ Enables stochastic policies, important for exploration and partial observability
- ☐ High variance due to Monte Carlo returns (no baseline)

4. Actor-Critic (A2C, PPO, TD3, SAC)

- ☐ **Actor** π_θ : updates the policy via gradient ascent
- ☐ **Critic** V_ϕ or Q_ϕ : provides low-variance targets (TD or advantage)
- ☐ Combines stochastic policies with improved learning signal

Why Do These Methods Still Fail?

- The same algorithm (e.g., PPO) can succeed or fail depending on the setup
- Performance is highly sensitive to:
 - ☐ environment
 - ☐ reward design
 - ☐ state representation

Core limitations:

- **Exploration:** poor data collection
 - ☐ e.g., sparse rewards in Montezuma's Revenge
- **Credit assignment:** delayed / weak learning signal
 - ☐ e.g., only reward at end of game
- **Representation:** insufficient state abstraction
 - ☐ e.g., partial observability in POMDPs

Performance is often limited by data, signal, and state – not the algorithm.

Exploration

Exploration: The Core Question

In reinforcement learning, exploration means collecting data that is **informative** and **reward-relevant**.

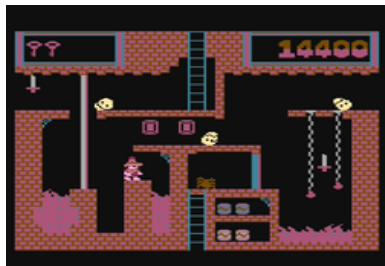
- Too little exploration: agent gets stuck in suboptimal behavior
- Too much random exploration: agent wastes samples
- Goal: explore the right states at the right time

Why Exploration Is Hard

- **Sparse rewards:** useful feedback is rare
- **High-dimensional state/action spaces:** combinatorial explosion of possibilities
- **Long horizons:** good decisions may pay off much later

Running example: Montezuma's Revenge

- Many rooms and objects
- Rewards appear only after long, precise action sequences



Baseline Exploration in Value-Based RL

Epsilon-greedy:

$$A_t = \begin{cases} \text{random action,} & \text{with probability } \epsilon \\ \arg \max_a Q(s_t, a), & \text{with probability } 1 - \epsilon \end{cases}$$

- Very simple and widely used
- Works in small or dense-reward tasks
- Limitation: **undirected** and **inefficient** in large spaces

Upper Confidence Bound in Bandits

UCB in bandits: optimism in the face of uncertainty

$$A_t = \arg \max_a \left(Q_t(a) + c \sqrt{\frac{\log t}{N_t(a)}} \right)$$

where $N_t(a)$ is the number of times action a has been taken.

- **Uncertainty-aware exploration** is better than pure randomness.
- Limitations in DRL:
 - ☐ Need uncertainty over large state-action spaces
 - ☐ No reliable confidence bounds with function approximation

Noisy Networks in Value Function Approximation

Noisy Networks: noisy output layers

$$y = (\mu^w + \sigma^w \odot \varepsilon^w) x + (\mu^b + \sigma^b \odot \varepsilon^b), \quad \varepsilon \sim \mathcal{N}(0, I)$$

- Noise parameters σ^w, σ^b are **learned** alongside mean parameters μ^w, μ^b
- Provides **state-dependent exploration**: explore more in uncertain regions
- Limitations:
 - ☐ Exploration remains **implicit** (no explicit uncertainty or bonus)
 - ☐ No guarantee of **directed exploration** (may still miss rare states)
 - ☐ Limited impact in very **sparse-reward** environments

Exploration via Action Noise (Continuous Control)

Action-space noise: add noise directly to the action

$$a_t = \pi_{\theta}(s_t) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

- Common in continuous control (DDPG, TD3)
- Simple way to ensure exploration in continuous action spaces
- Noise is typically **fixed** (not learned)
- Limitations:
 - ☐ Independent across time → jittery, inconsistent behavior
 - ☐ Not **state-dependent** → same noise everywhere
 - ☐ No notion of **uncertainty or direction**

Entropy Regularization in Policy Methods

Entropy Maximization: encourage stochastic policies by maximizing:

$$J_{\text{ent}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t (r_t + \alpha \mathcal{H}(\pi_{\theta}(\cdot | s_t))) \right]$$

- Encourages **stochastic policies** (exploration as objective)
- PPO: entropy bonus in the objective
- SAC: maximum-entropy RL as a core principle
- Limitations:
 - ☐ No notion of **uncertainty or direction**
 - ☐ Encourages randomness, not goal-directed exploration

What Is Missing?

Baseline methods are mostly:

- random (epsilon-greedy, entropy)
- weakly structured (noise-based methods)

They do not answer:

Where should we explore next?

Key idea: exploration should be directed and informed.

Three guiding principles:

- **Uncertainty:** explore what we do not know
- **Novelty:** explore what we have not seen
- **Surprise:** explore what we cannot predict

Uncertainty-Based Exploration

What uncertainty-driven exploration means

- We are uncertain about values/models because data is limited
- This is **epistemic uncertainty** (reducible with more data)
- Exploration should target state-action pairs with high epistemic uncertainty

Formal view:

learn $p(Q \mid \mathcal{D}_t)$ (intractable in deep RL)

Two canonical decision rules:

- **Optimism/UCB**: act on upper confidence estimate
- **Posterior sampling (Thompson)**: sample one hypothesis and act greedily

UCB vs Thompson Sampling in Bandits

- **UCB:** choose action with highest upper confidence bound

$$a_t = \arg \max_a [\mu_t(a) + \beta \sigma_t(a)]$$

- **Thompson Sampling:** sample a value estimate from the posterior and act greedily

$$\tilde{q} \sim p(q \mid \mathcal{D}_t), \quad a_t = \arg \max_a \tilde{q}(a)$$

- UCB is deterministic
- TS is probabilistic and naturally trades off exploration/exploitation
- In practice, both require an uncertainty estimate

From Exact Posteriors to Deep RL

Problem in deep RL:

- No tractable posterior over neural-network $Q_{\theta}(s, a)$
- Need scalable approximations to represent uncertainty

Approximate posterior samples:

$$\{Q^{(j)}\}_{j=1}^K \approx \text{samples from } p(Q \mid \mathcal{D})$$

- $Q^{(j)}$ is one value-function hypothesis (network or head)
- Uncertainty proxy at (s, a) : $\text{Var}_j [Q^{(j)}(s, a)]$

Ensembling

- Initialize K Q-functions differently: $Q^{(1)}, \dots, Q^{(K)}$ (and target networks)
- Action selection during data collection:
 - ☐ mean policy: $\arg \max_a \frac{1}{K} \sum_{j=1}^K Q^{(j)}(s, a)$
 - ☐ UCB-style: mean + bonus from variance
 - ☐ Thompson-style: sample acting head k_{act} and act greedily
- Training:
 - ☐ Train all on the same replay buffer $\mathcal{D}$
 - ☐ Update each head with its own TD targets
- Limitation:
 - ☐ Same data can make hypotheses collapse
 - ☐ Uncertainty may be poorly calibrated

Bootstrap ensembles

- Initialize K Q-functions: $Q^{(1)}, \dots, Q^{(K)}$ (and target networks)
- Data collection:
 - ☐ Sample acting head once per episode: $k_{\text{act}} \sim \text{Unif}(\{1, \dots, K\})$
 - ☐ Act with that head for whole episode: $a_t = \arg \max_a Q^{(k_{\text{act}})}(s_t, a)$
 - ☐ Store all transitions in shared replay buffer $\mathcal{D}$
 - ☐ At insertion, assign fixed bootstrap mask $m_t \in \{0, 1\}^K$ to each transition
- Training:
 - ☐ Head j trains only on transitions with $m_t^{(j)} = 1$
 - ☐ k_{act} and m_t are sampled separately
- Benefits:
 - ☐ Different data views keep hypotheses diverse for longer
 - ☐ Episode-wise commitment yields deep exploration (multi-step)

Randomized priors

- In sparse-data regions, (bootstrap) ensembles may still become overconfident
- Add fixed random prior function $p^{(j)}(s, a)$ to each head:

$$\hat{Q}^{(j)}(s, a) = Q^{(j)}(s, a) + \lambda p^{(j)}(s, a)$$

- Train only $Q^{(j)}$; keep $p^{(j)}$ fixed
- Act with $\hat{Q}^{(j)}$ (UCB or Thompson-style head sampling)
- Uncertainty is sustained away from observed data; improves exploration in sparse-reward tasks

Novelty-Based Exploration

- Encourage visiting **novel or rare states**

General approach:

$$r_t = r_t^{\text{ext}} + \beta r_t^{\text{int}}$$

- r_t^{ext} : external reward from the environment
- r_t^{int} : **intrinsic reward** (novelty signal of state or state-action pair)
- β : scale of exploration pressure

Count-Based Exploration

Tabular intuition:

$$r_t^{\text{int}}(s_t) = \frac{1}{\sqrt{N(s_t)}} \quad \text{or} \quad r_t^{\text{int}}(s_t, a_t) = \frac{1}{\sqrt{N(s_t, a_t)}}$$

- High bonus for rarely visited states
- Bonus decays as counts increase

Deep RL challenge:

- Exact counts are impossible in high-dimensional observations
- Approximation strategies:
 - ☐ **Density models**: estimate $p(s)$ and derive pseudo-counts
 - ☐ **Hashing**: map states to discrete bins

Bellemare et al., 2016; Tang et al., 2016

Curiosity-Based Exploration

- Instead of counting visits, measure how **surprising** a transition is
- Surprise reflects how much the agent's model fails to predict the world

General idea:

$$r_t^{\text{int}} \propto \text{prediction error or model improvement}$$

- **Prediction error** (RND, ICM): proxy for novelty
 - ☐ High surprise $\rightarrow$ novel or poorly understood state
 - ☐ Low surprise $\rightarrow$ familiar and predictable state
- **Information gain** (VIME): change in model belief
 - ☐ Harder to implement in deep RL – not covered in this course

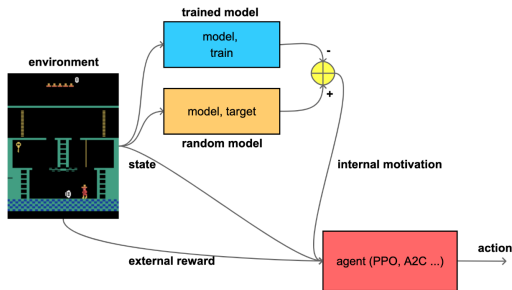
Random Network Distillation (RND)

- Fix a random target network f_{target}
- Train a predictor network f_{θ} to match it

Intrinsic reward:

$$r_t^{\text{int}} = \|f_{\theta}(s_t) - f_{\text{target}}(s_t)\|^2$$

- Unseen states $\rightarrow$ high error
- Frequent states $\rightarrow$ low error
- No dynamics model required
- Stable and simple to implement



Burda et al., 2018

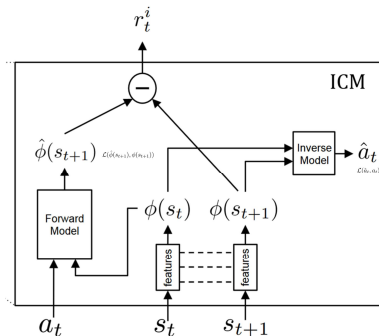
Intrinsic Curiosity Module (ICM)

- Learn a feature representation $\phi(s)$ of states
- Train a **forward model** to predict next features: $\hat{\phi}(s_{t+1}) = f(\phi(s_t), a_t)$

Intrinsic reward:

$$r_t^{\text{int}} = \|\hat{\phi}(s_{t+1}) - \phi(s_{t+1})\|^2$$

- High error $\rightarrow$ surprising transition
- Low error $\rightarrow$ predictable dynamics



- Encourages exploration of transitions that are hard to predict

Disagreement-Based Exploration

- Use multiple forward models $\{f^{(k)}\}_{k=1}^K$
- Each predicts next state (or features):

$$\hat{s}_{t+1}^{(k)} = f^{(k)}(s_t, a_t)$$

Intrinsic reward from disagreement:

$$r_t^{\text{int}} \propto \text{Var}_k[\hat{s}_{t+1}^{(k)}]$$

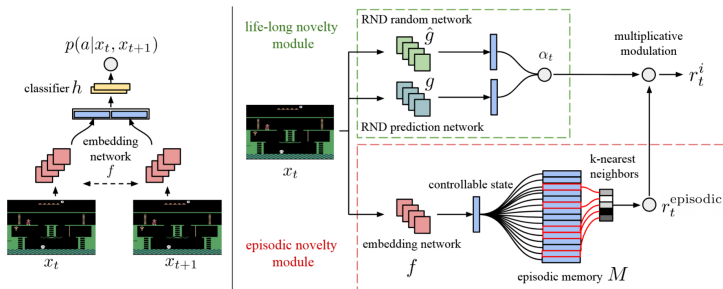
- High disagreement $\rightarrow$ uncertain / poorly understood dynamics
- Low disagreement $\rightarrow$ consistent predictions
- Captures epistemic uncertainty in dynamics
- More robust than single-model prediction error

Combining Intrinsic Signals

- Combine multiple novelty signals for stronger exploration

For example, the Never-Give-Up (NGU) method:

- **Episodic novelty**: how often visited in current episode
- **Lifelong novelty**: how familiar across training (RND-like)



Badia et al., 2020

Novelty Methods: Strengths and Failure Modes

- Works well in [sparse-reward settings](#)
- Easy to integrate via intrinsic reward:

$$r_t = r_t^{\text{ext}} + \beta r_t^{\text{int}}$$

- Limitations:
 - ☐ [Noisy-TV problem](#): stochastic but irrelevant observations remain novel
 - ☐ Intrinsic reward does not distinguish useful vs irrelevant novelty
 - ☐ Non-stationary intrinsic rewards $\rightarrow$ unstable value estimates
 - ☐ Large β can shift behavior toward novelty instead of task reward
- Some remedies:
 - ☐ Normalize or clip intrinsic rewards (control scale)
 - ☐ Schedule or anneal β (shift toward exploitation)
 - ☐ Combine episodic and lifelong novelty signals (e.g., NGU)

Exploration: Summary

Where should we explore next?

Exploration can be introduced at different levels:

■ Action selection:

- ☐ ϵ -greedy, UCB, Thompson sampling
- ☐ action noise (continuous control)

■ Model / value estimation:

- ☐ Noisy networks
- ☐ Ensembles, bootstrap, randomized priors

■ Reward shaping (intrinsic motivation):

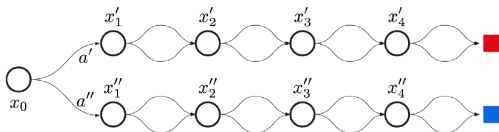
- ☐ Count-based, curiosity (ICM, RND), disagreement

■ Effective exploration requires structure and direction

Credit Assignment

What Is the Credit Assignment Problem?

- **Core question:** Which past actions caused the current outcome?
- Most real problems have:
 - ☐ **Delayed** and **sparse** feedback
 - ☐ Many irrelevant intervening decisions



- Hard to separate **informed decisions** from **serendipitous outcomes**
- Credit $\neq$ value: credit captures the **influence of a specific action** on a specific outcome

Three Root Causes of Hard Credit Assignment

■ **Depth** — large temporal distance between action and outcome

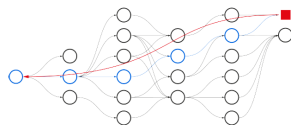
- TD error shrinks exponentially; recency bias assigns credit to wrong actions

■ **Density** — low action influence (sparse rewards)

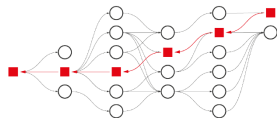
- Rare goal states; most TD errors carry no information

■ **Breadth** — many alternative paths to the same outcome

- Credit dilutes across equivalent trajectories; no clear bottleneck action



(a) Depth of the MDP.



(b) Density of the MDP.



(c) Breadth of the MDP.

A Taxonomy of Credit Assignment Methods

Class	Key Idea	Challenge
Time contiguity	TD / eligibility traces	Depth
Return decomposition	RUDDER, RRD	Depth
Hindsight / future cond.	HER, HCA	Density
Auxiliary goals	GVF, UVFA	Density
Sequence modelling	Decision Transformer	Depth + Density
Backward planning	Recall traces	Depth
Meta-learning	Meta-gradient	Depth

- No single method solves all credit assignment problems

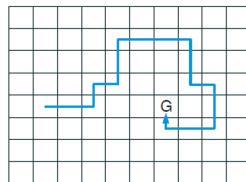
Time Contiguity: TD Learning

One-step TD target:

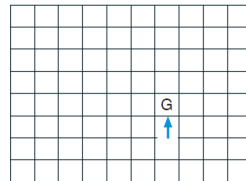
$$y_t^{(1)} = r_t + \gamma V(s_{t+1})$$

- TD has **local credit only**: reward moves back one step per update
- Slow propagation in long-horizon problems
- Leads to **recency bias**: actions close to reward get too much credit

Path taken



Action values increased by one-step Sarsa



Time Contiguity: n-Step, λ -Returns, and GAE

n-step target:

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V(s_{t+n})$$

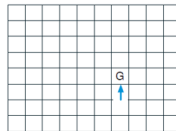
λ -return:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} y_t^{(n)}$$

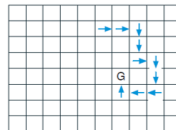
Policy-gradient analogue (GAE):

$$\hat{A}_t^{\text{GAE}(\lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

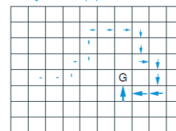
Action values increased
by one-step Sarsa



Action values increased
by 10-step Sarsa



Action values increased
by Sarsa(λ) with $\lambda=0.9$



■ Larger n or λ speeds up propagation, but increases variance

Breadth: Reducing Credit Noise with Advantages

- Problem: many actions do not influence the outcome $\Rightarrow$ noisy credit assignment
- Solution: compare action to a baseline (expected outcome)

$$A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$$

- Only assign credit when action is **better or worse than expected**
 - ☐ reduces variance from irrelevant actions
 - ☐ focuses learning on **decision-relevant deviations**
- Complements TD: TD handles **when**, advantage handles **whether an action mattered**

Return Decomposition (RUDDER)

- Problem: reward arrives **too late** to identify the causal action
- Key idea: **move the reward to the important event**
- Learn a return predictor:

$$\hat{G}_t \approx \text{future return}$$

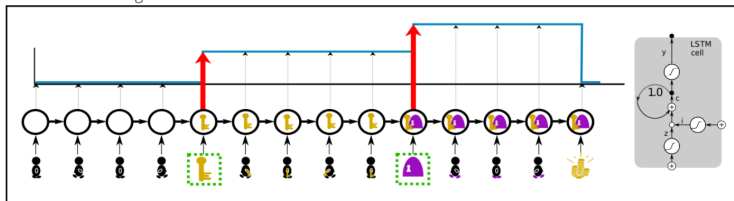
- Redistribute reward:

$$\tilde{r}_{t+1} = \hat{G}_{t+1} - \hat{G}_t$$

- Large change in prediction $\Rightarrow$ important event
- Turns long-horizon problem into **local learning**

RUDDER: Using Sequence Models

- Uses a **sequence model** (e.g., LSTM) to predict return from trajectory
 - Input: $(s_0, a_0, \dots, s_t)$
 - Output: predicted return $\hat{G}_t$
- Credit is assigned where prediction changes most
 - identifies **key events** in long sequences
- Cost: additional model + training complexity



Hindsight Experience Replay (HER)

Key idea: learn from failed trajectories

- Setting: goal-conditioned RL — reward depends on goal g
- Problem: goal not reached $\Rightarrow$ no learning signal
- Key idea: every failed trajectory succeeded at some other goal
 - take achieved state $g' = s_T$
 - relabel transitions: $(s_t, a_t, r_t, g) \rightarrow (s_t, a_t, r'_t, g')$
 - $r'_t = 1$ if $s_t = g'$, else 0
- Over many episodes: agent learns to reach many subgoals $\Rightarrow$ eventually reaches original goal
- Addresses density: turns sparse failure signal into dense training data
- Limitation: requires a goal-conditioned MDP

Andrychowicz et al., 2017

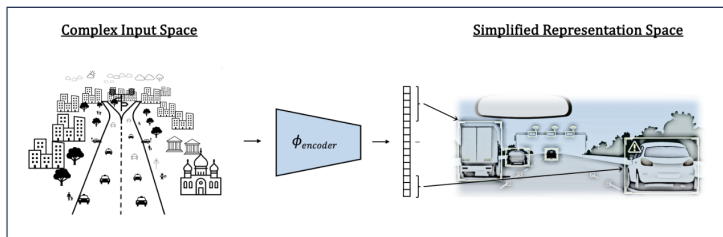
Credit Assignment: Practical Takeaways

- **Mild delay:** TD, n -step, GAE (default tools)
- **Long delay:** return decomposition (RUDDER)
- **Sparse rewards:** hindsight (HER)
- Diagnose the bottleneck: **depth vs density vs breadth**
- Strong link to **representation learning**:
 - no good features $\Rightarrow$ no correct credit assignment

State Representation Learning

Why State Representation Learning?

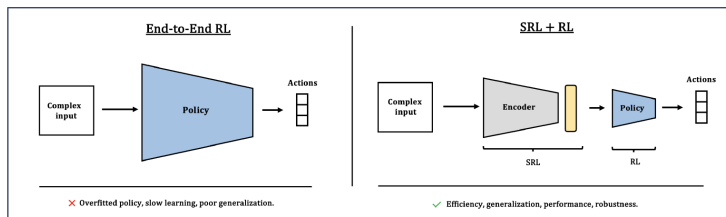
- Raw observations are **high-dimensional, noisy, and often partially observable**
- SRL learns a latent state that keeps **control-relevant** information



Goal: compress without losing decision-relevant structure

End-to-End RL vs SRL + RL

- **End-to-end:** policy/value learns features and control jointly
- **SRL + RL:** encoder learns reusable latent structure, RL acts on latent states
 - Better modularity, often better sample efficiency and robustness



Desiderata

■ Structure

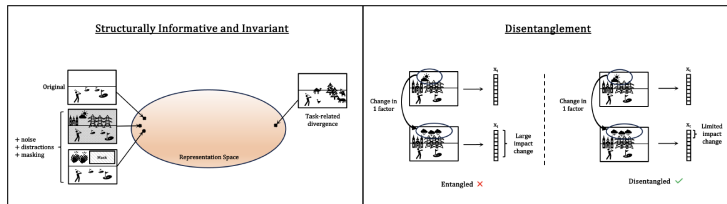
- the latent space should reflect causal and task-relevant relationships in the environment, not irrelevant variation in the observation

■ Continuity

- nearby states in the environment should map to nearby points in latent space
- small observation changes should produce small representation changes

■ Sparsity

- only a small number of latent dimensions should be active per state
- promotes disentanglement and reduces spurious correlations



Taxonomy of SRL Methods

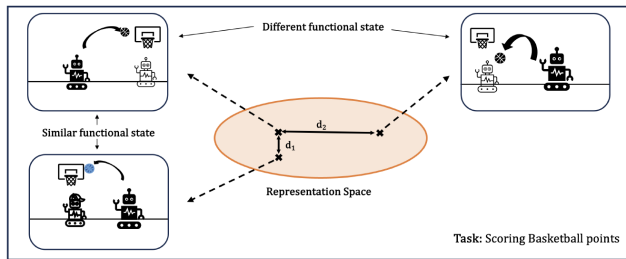
- Six main classes in online model-free SRL:

- ☐ Metric-based
- ☐ Auxiliary-task-based
- ☐ Data augmentation-based
- ☐ Contrastive
- ☐ Non-contrastive
- ☐ Attention-based

- These are not mutually exclusive: many modern methods are hybrids

Metric-Based SRL and Bisimulation

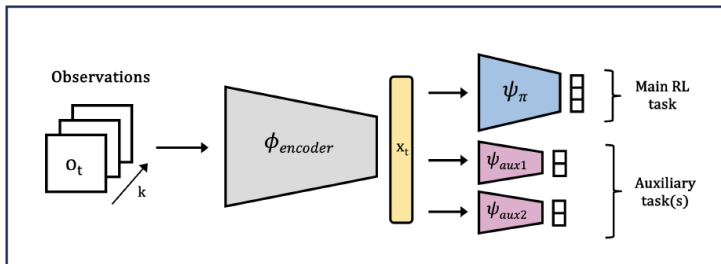
- Shape latent distances to match task-relevant behavioral similarity
- **Bisimulation**: states are close if rewards and transition dynamics are similar



- Strength: principled abstraction
- Limitation: metric computation and stability can be expensive

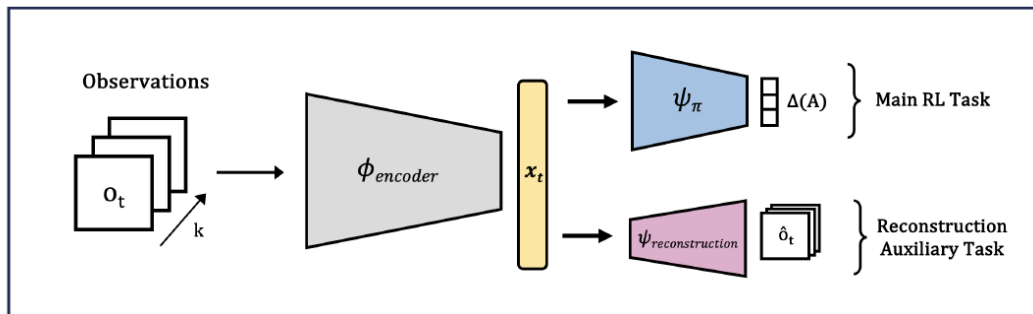
Auxiliary Tasks

- Add **auxiliary** side objectives (reconstruction, forward/inverse dynamics, reward prediction)

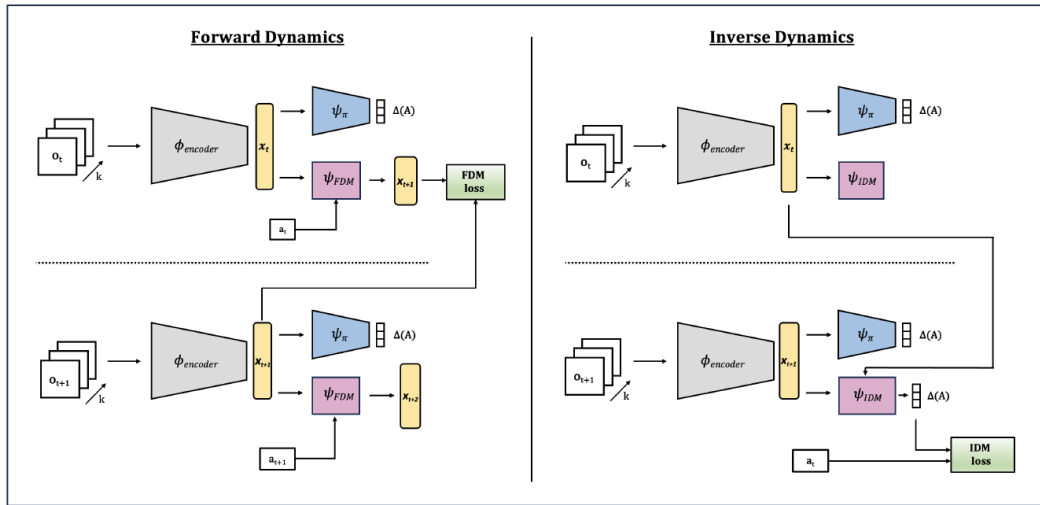


- Benefits: richer signal under **sparse reward**
- Risk: auxiliary objective may not align with control objective

Auxiliary Tasks: Reconstruction

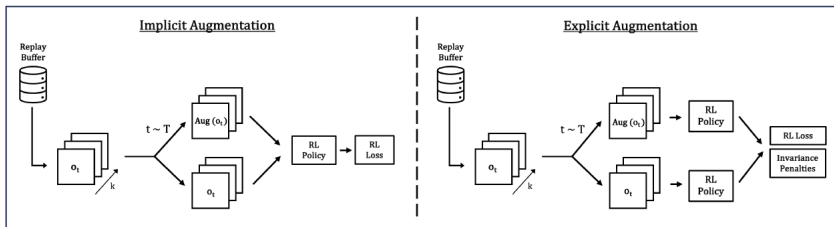


Auxiliary Tasks: Dynamics Prediction

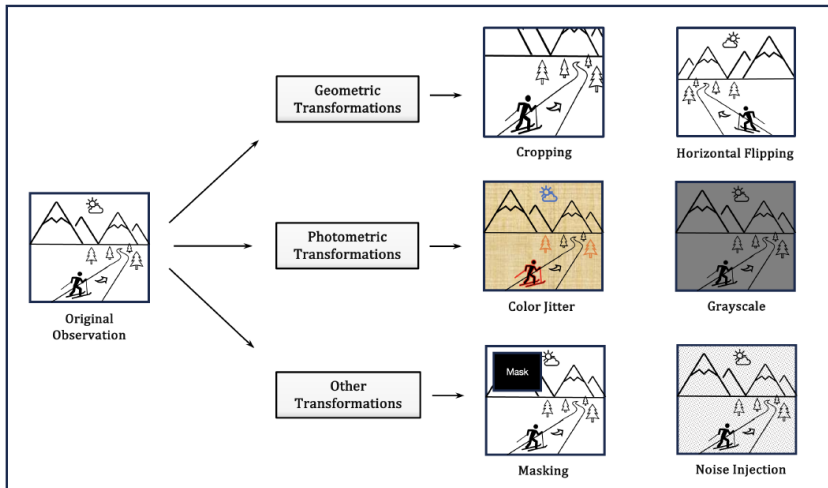


Data Augmentation for Invariance

- Encourage **invariance to nuisance** visual transformations
- Usually improves generalization and data efficiency
- Strong augmentations can remove task-critical cues if misconfigured

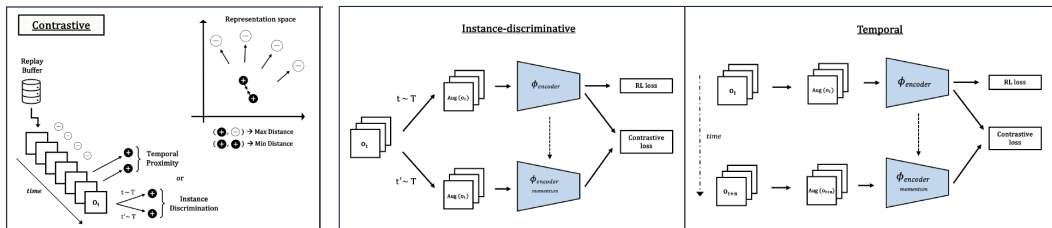


Common Data Augmentation Techniques



Contrastive SRL

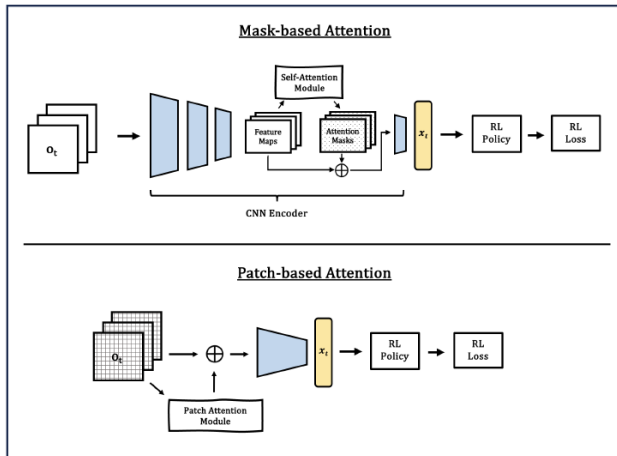
- Pull together representations of similar states, push apart different ones



$$\mathcal{L}_{\text{NCE}} = -\mathbb{E}_{(o, o^+, \{o_i^-\})} \left[\log \frac{\exp(\text{sim}(\phi_\theta(o), \phi_\theta(o^+)))}{\exp(\text{sim}(\phi_\theta(o), \phi_\theta(o^+))) + \sum_{i=1}^N \exp(\text{sim}(\phi_\theta(o), \phi_\theta(o_i^-)))} \right]$$

Attention-Based SRL

- Learn attention masks over spatial/temporal features
- Focus on decision-relevant regions and suppress distractors
- Improves interpretability but adds architectural complexity



Summary

Summary: Core Challenges in RL

- **Exploration:** how to efficiently discover rewarding behaviors in large, sparse, or deceptive environments
- **Credit assignment:** how to attribute outcomes to past actions across long horizons
- **State representation learning:** how to learn compact, informative latent states from high-dimensional observations

These challenges are deeply interconnected:

- Better representations can improve both exploration and credit assignment
- Effective exploration can provide the data needed for learning good representations and accurate credit signals
- Accurate credit assignment can guide representation learning toward decision-relevant features